

```

1  import components.naturalnumber.NaturalNumber;
10
11 /**
12  * Program to evaluate XMLTree expressions of {@code int}.
13  *
14  * @author Eric Zhou
15  *
16  */
17 public final class XMLTreeNNEvaluationEvaluator {
18
19     /**
20      * Private constructor so this utility class cannot be instantiated.
21      */
22     private XMLTreeNNEvaluationEvaluator() {
23     }
24
25     /**
26      * Evaluate the given expression.
27      *
28      * @param exp
29      *         the {@code XMLTree} representing the expression
30      * @return the value of the expression
31      * @requires * [exp is a subtree of a well-formed XML arithmetic expression]
32      *            and [the label of the root of exp is not "expression"]
33      *
34      * @ensures evaluate = [the value of the expression]
35      */
36     private static NaturalNumber evaluate(XMLTree exp) {
37         assert exp != null : "Violation of: exp is not null";
38
39         NaturalNumber result;
40
41         if (exp.label().equals("number")) { //decide the end of recursion
42             result = new NaturalNumber1L(Integer.parseInt(exp.attributeValue("value")));
43         } else {
44             NaturalNumber left = evaluate(exp.child(0)); //recursion
45             NaturalNumber right = evaluate(exp.child(1));
46             result = new NaturalNumber1L(left);
47             String op = exp.label(); // check the operator
48
49             if (op.equals("plus")) {
50                 result.add(right);
51             } else if (op.equals("minus")) {
52                 if (result.compareTo(right) < 0) { //assert the number should be positive
53                     Reporter.fatalErrorToConsole("NaturalNumber should be positive");
54                 }
55                 result.subtract(right);
56             } else if (op.equals("times")) {
57                 result.multiply(right);
58             } else if (op.equals("divide")) {
59                 if (right.isZero()) { // assert that cannot divide zero
60                     Reporter.fatalErrorToConsole("cannot divide by zero");
61                 }
62                 result.divide(right);
63             }
64         }
65         return result;
66     }
67
68     /**
69      * Main method.
70      *
71      * @param args
72      *         the command line arguments
73      */

```

```
74 public static void main(String[] args) {
75     SimpleReader in = new SimpleReader1L();
76     SimpleWriter out = new SimpleWriter1L();
77
78     out.print("Enter the name of an expression XML file: ");
79     String file = in.nextLine();
80     while (!file.equals("")) {
81         XMLTree exp = new XMLTree1(file);
82         out.println(evaluate(exp.child(0)));
83         out.print("Enter the name of an expression XML file: ");
84         file = in.nextLine();
85     }
86
87     in.close();
88     out.close();
89 }
90
91 }
92
```